

```

1. def generate_signals(data):
2.     """Generate trading signals without look-ahead bias"""
3.     # Create a copy of the data to avoid modifying the original
4.     df = data.copy()
5.
6.     # Calculate indicators using only past data
7.     df['SMA20'] = df['Adj Close'].rolling(window=20).mean()
8.     df['SMA50'] = df['Adj Close'].rolling(window=50).mean()
9.
10.    # Initialize signal column
11.    df['Signal'] = 0
12.
13.    # Generate signals - note the shift(1) to avoid look-ahead bias
14.    # We only know today's moving averages after the close
15.    df['Signal'] = np.where(
16.        df['SMA20'].shift(1) > df['SMA50'].shift(1), 1, 0
17.    )
18.
19.    return df
20.

```

Note the use of `shift(1)` when generating signals. This ensures we're only using information that would have been available at the time of the trade, not peeking into the future.

Strategy Implementation: From Concept to Code

With clean data in hand, the next step is implementing the trading strategy in a way that accurately reflects how it would behave in live trading. A robust implementation should account for:

1. **Entry and exit rules:** When to enter and exit positions
2. **Position sizing:** How much capital to allocate to each trade
3. **Risk management:** Stop-loss levels, maximum position sizes, etc.